Tutoriales

- Java
- C/C++ de Linux
- Metodologías y diseño orientado a objetos
- CSS

Enlaces

- Chuwiki
- Diario de Programación
- Foro de Java y C++
- Pasatiempos

Licencia

Esta obra está bajo una licencia de Creative Commons.
Para reconocer la autoría debes poner el enlace https://old.chuidiang.org

Ejemplo sencillo con JavaHelp

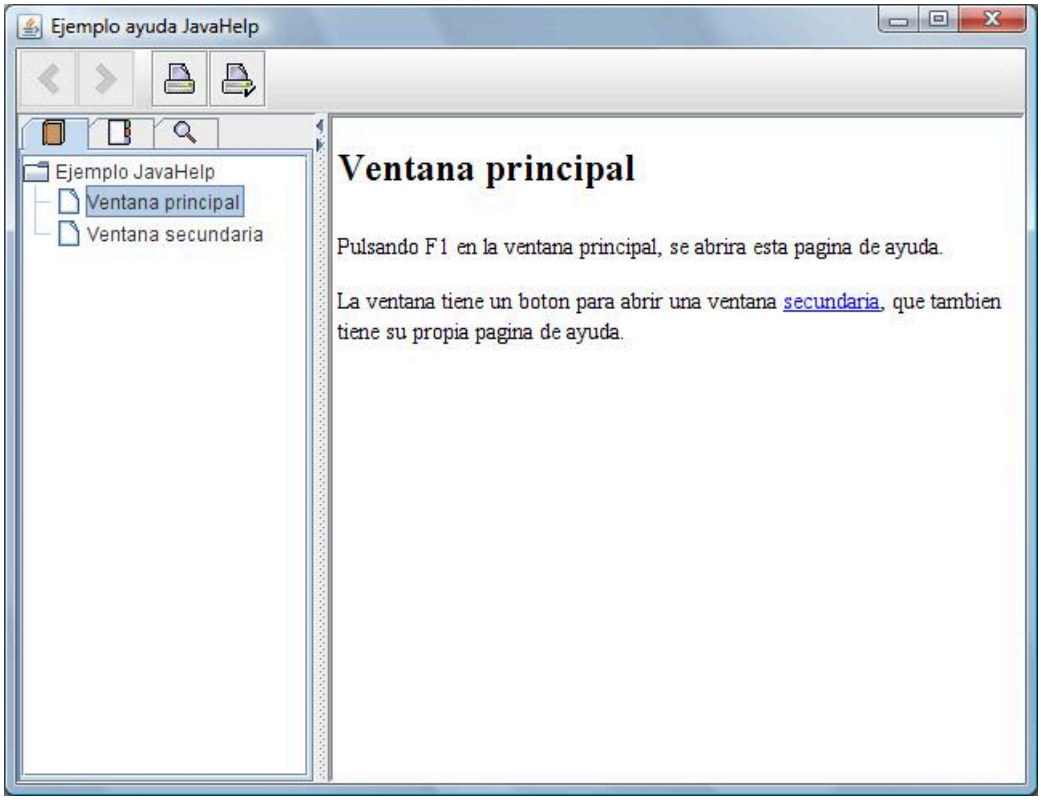
JavaHelp es una librería opcional de *Java* que nos permite poner ventanas de ayuda a nuestras aplicaciones de forma sencilla. Dichas ventanas serán como las que estamos acostumbrados en cualquier aplicación. Si en el mismo navegador en que estás leyendo esto pulsas F1, verás una ventana de ayuda, similar a la que podrás visualizar con *JavaHelp*.

En este tutorial vamos a ver un ejemplo sencillo con *JavaHelp*. No vamos a complicarnos la vida, sino símplemente lo mínimo imprescindible para empezar a trabajar.

Haremos una pequeña aplicación tonta, que no hará nada. Tendrá una ventana principal con un menú de "Ayuda" y un botón que despliegue una ventana secundaria. La ventana secundaria tampoco tendrá nada especial, un JLabel y un JTextField, por aquello de poner algo. Por supuesto, la ayuda de *JavaHelp* será cortita, no hay mucho que explicar en una ventana que no hace nada.

En esa aplicación añadiremos la ayuda con *JavaHelp*. Pondremos una ventana de ayuda principal que se abrirá al pulsar "Ayuda" en el menú y otras dos ventanas de ayuda que se abrirán al pulsar, respectivamente, F1 en cada una de las ventanas.

La ventana de ayuda que obtendremos será similar a la de la figura:



Bajarse JavaHelp

Lo primero, por supuesto, es bajarse *JavaHelp* y ponerlo en algún sitio. Se puede bajar *JavaHelp* de la página de Sun. Al bajarlo e instalarlo, tendremos los jar necesarios para nuestra aplicación junto con algunas herramientas, algunas de ellas también necesarias para preparar la ayuda.

Directorio para la ayuda

Todos los ficheros de ayuda, tanto de configuración de JavaHelp como los que hagamos con nuestros textos de ayuda, necesitamos meterlos más o menos juntos en un directorio. Una opción es crear en

nuestro proyecto un directorio *help*. Dentro de él podemos poner los ficheros de configuración de JavaHelp y un subdirectorio *html*. En el subdirectorio html irán en formato *html* todos los ficheros de ayuda que nosotros queramos mostrar y que, por supuesto, tendremos que escribir.

Podemos poner esta estructura como queramos, pero esta que hemos mencionado es la que haremos en nuestro ejemplo. Lo que sí es importante es que los ficheros html de nuestra ayuda estén en directorios por debajo de donde estén nuestros ficheros de configuración de *JavaHelp*. *JavaHelp* no permite que vayamos a otros directorios superiores a buscar la ayuda.

Ficheros html

Los ficheros html con la ayuda tendremos que escribirlos nosotros. Debemos escribir tantos como pantallas o temas queramos que tenga nuestra ayuda. Cada fichero html que hagamos se mostrará completo en pantalla, por lo que si queremos varias pantallas de ayuda, debemos hacer varios ficheros html.

En nuestra aplicación tonta, haremos tres ficheros html, correspondientes a la pantalla principal de la ayuda, que cuenta lo que hace nuestra aplicación, la ayuda de la ventana principal y la ayuda de la ventana secundaria. Debajo de nuestro directorio html, crearemos los ficheros main.html, principal.html y secundaria.html. Como vemos en main.html, se pueden poner enlaces entre ellos usando paths relativos.

Fichero mapa de JavaHelp

El siguiente paso es crear un fichero mapa de *JavaHelp*. Este fichero no es más que un fichero XML en el que se da una "clave" a cada uno de los ficheros html que creamos. Es decir, a cada fichero html se le da un nombre, que servirá para identificarlo a partir de ahora. Tanto el resto de los ficheros de configuración de JavaHelp como nuestro código Java referenciarán a los ficheros .html con el nombre que pongamos en este fichero mapa.

El fichero mapa tiene extensión .jhm -Java Help Map-, y en nuestro ejemplo será map_file.jhm. Lo colocaremos debajo del directorio help, en parelelo con el subdirectorio html.

El contenido es más o menos así

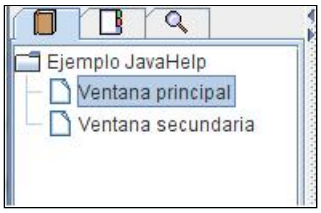
```
<?xml version='1.0' encoding='ISO-8859-1'?>
<!DOCTYPE map
PUBLIC "-//Sun Microsystems Inc.//DTD JavaHelp Map Version 2.0//EN"
"http://java.sun.com/products/javahelp/map_2_0.dtd">
<map version="1.0">
   <mapID target="aplicacion" url="html/main.html" />
   <mapID target="ventana_principal" url="html/principal.html" />
   <mapID target="ventana_secundaria" url="html/secundaria.html" />
</map>
```

Aparte de la cabecera del XML, vemos el tag principal *<map>* y dentro de él varios tag *<mapID>*. Cada *<mapID>* corresponde a un fichero html de ayuda. *target* es la "clave" o "nombre" que daremos al fichero html para referenciarlo a partir de este momento. *url* es el fichero html en sí. El path del fichero html es relativo a la ubicación de este fichero mapa, por ello llevan delante el subdirectorio html en el que hemos metido todos nuestros ficheros .html.

Vamos con el resto de ficheros. No es obligatorio hacerlos si no queremos que nuestra ayuda no tenga la pestaña correspondiente.

Fichero de Tabla de Contenidos de JavaHelp

Aunque posiblemente no es obligatorio, lo normal es que nuestra ayuda tenga una "tabla de contenidos". Esta tabla de contenidos no es más que una especie de "árbol" en la que van los capítulos y subcapitulos de nuestra ayuda. En nuestro ejemplo, esta tabla de contenidos tendrá un único elemento principal con la ayuda de nuestra aplicación. Este elemento principal tendrá dos subapartados, uno para la ventana principal y otro para la secundaria. Vaya, más o menos lo de la figura



Los textos para este árbol, así como el fichero de ayuda que debe mostrarse al seleccionar uno de los item del árbol, se ponen en un fichero XML que luego leerá JavaHelp. Este fichero tiene extensión .xml y para nuestro ejemplo será toc.xml. El contenido de este fichero es el siguiente

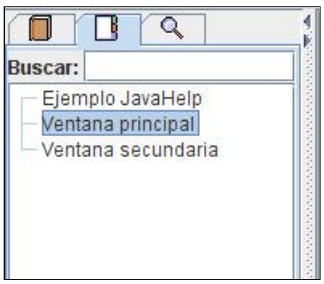
```
<?xml version="1.0" encoding="ISO-8859-1"?>
<toc version="1.0">
   <tocitem text="Ejemplo JavaHelp" target="aplicacion">
      <tocitem text="Ventana principal" target="ventana_principal"/>
      <tocitem text="Ventana secundaria" target="ventana_secundaria"/>
   </tocitem>
</toc>
```

Además del encabezado de XML, vemos como tag principal *<toc>* y luego varios tags *<tocitem>* que podemos ir anidando como queramos que se muestre más adelante el árbol. En cada *<tocitem>* ponemos un *text* para indicar el texto visible y un *target*, para indicar la "clave" o "nombre" del fichero html de ayuda que queremos mostrar cuando se seleccione este item en el árbol. Estas "claves" o "nombres" son las que dimos anteriormente en el fichero mapa de *JavaHelp*.

Este fichero *tox.xml* lo colocaremos en el directorio *help*, junto con el fichero de mapa *map_file.jhm*.

Fichero índice de JavaHelp

Otra pestaña adicional que puede tener nuestra ventana de ayuda es un índice alfabético por temas, como el de la figura.



Este índice, cómo no, se crea por medio de otro fichero XML que tenemos que hacernos a mano. El fichero tiene extensión xml y para nuestro ejemplo puede ser como indice.xml. El contenido de este fichero es

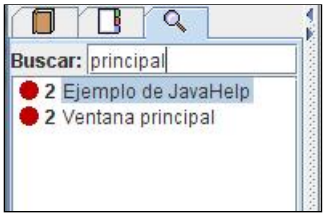
```
<?xml version='1.0' encoding='ISO-8859-1' ?>
<!DOCTYPE index
PUBLIC "-//Sun Microsystems Inc.//DTD JavaHelp Index Version 1.0//EN"
"http://java.sun.com/products/javahelp/index_1_0.dtd">
<index version="1.0">
   <indexitem text="Ejemplo JavaHelp" target="aplicacion"/>
   <indexitem text="Ventana principal" target="ventana_principal"/>
   <indexitem text="Ventana secundaria" target="ventana_secundaria"/>
</index>
```

Aparte de la cabecera XML, vemos un tag principal *<index>* y luego varios *<indexitem>*. Cada uno de estos *<indexitem>* serán los que se vean en la ventana. text es el texto que se mostrará y target la clave del fichero html que se visualizará cuando seleccionemos en la ventana el *<indexitem>*.

Este fichero *index.xml* lo colocaremos junto con los dos anteriores, el de *map_file.jhm* y *toc.xml*.

Búsqueda en la ayuda de JavaHelp

Otra pestaña opcional de *JavaHelp* que podemos colocar, es la de búsqueda. Poniendo esta pestaña, nos saldrá una caja de búsqueda por la que podremos buscar palabras dentro de la ayuda. La pestaña será como la de la foto



Para esta ventana no necesitamos crear a mano ningún fichero especial. Lo que sí necesitamos es ejecutar una de las herramientas que viene con JavaHelp. Esta herramienta se llama jhindexer.jar y lo que hace es revisar nuestros ficheros html de ayuda y generar una pequeña base de datos que sirva para hacer las búsquedas más rápidas y eficientes. Si más adelante tocamos los ficheros html, debemos volver a ejecutar esta herramienta.

Para ejecutarla, vamos al directorio help y ejecutamos

```
$ cd /PATH_PROYECTO/help
$ java -jar /PATH_JAVAHELP/bin/jhindexer.jar html
```

y en mi caso concreto, que estoy en Windows y para mi aplicación, con mis paths concretos y demás...

```
c:\> cd c:\proyectos\ejemplo_java_help\help
c:\> java -jar C:\Aplicaciones\jh2.0\javahelp\bin\jhindexer.jar html
```

es decir, ejecutamos *jhindexer.jar* pasándole como parámetro el directorio donde están nuestros html. Esto creará un directorio JavaHelpSearch dentro del cual estará la base de datos creada por esta herramienta. Mejor no tocarla....

Mas pestañas en JavaHelp

Hay posibilidad de poner más pestañas, como una pestaña de favoritos y demás, pero de momento creo que vale.

Fichero principal de configuración de JavaHelp

Hasta aquí los ficheros "secundarios" de JavaHelp. Ahora necesitamos hacer el fichero principal, el que lo configura todo. Este fichero, para variar, va en formato XML, pero se le pone extensión .hs -Help Set- Para nuestro ejemplo, el fichero será help_set.hs.

El contenido de este fichero es más o menos

```
<?xml version="1.0" encoding='ISO-8859-1' ?>
<!DOCTYPE helpset
PUBLIC "-//Sun Microsystems Inc.//DTD JavaHelp HelpSet Version 1.0//EN"
"http://java.sun.com/products/javahelp/helpset_1_0.dtd">
<helpset version="1.0">
   <title>Ejemplo ayuda JavaHelp</title>
   <maps>
      <!-- Pagina por defecto al mostrar la ayuda -->
      <homeID>aplicacion</homeID>
      <!-- Que mapa deseamos -->
      <mapref location="map_file.jhm"/>
   </maps>

   <!-- Las Vistas que deseamos mostrar en la ayuda -->
   <!-- La tabla de contenidos -->
   <view>
      <name>Tabla Contenidos</name>
      <label>Tabla de contenidos</label>
```

```
      <type>javax.help.TOCView</type>
      <data>toc.xml</data>
   </view>

   <!-- El indice -->
   <view>
      <name>Indice</name>
      <label>El indice</label>
      <type>javax.help.IndexView</type>
      <data>indice.xml</data>
   </view>

   <!-- La pestana de busqueda -->
   <view>
      <name>Buscar</name>
      <label>Buscar</label>
      <type>javax.help.SearchView</type>
      <data engine="com.sun.java.help.search.DefaultSearchEngine">
         JavaHelpSearch
      </data>
   </view>

</helpset>
```

El tag principal es *<helpset>*.

Dentro de él hay tres tipos de tags.

- Primero un *<title>*, que es el título de nuestra ventana de ayuda.
- Después uno obligatorio que dice dónde está el fichero mapa, el tag *<maps>*.
- Luego podemos poner varios tags *<view>*, cada uno correspondiente a una de las pestañas que queramos mostrar. Si no queremos pestaña, no hace falta poner el *<view>* correspondiente.

El tag <maps> lleva:

- Un *<homeID>* con la clave del fichero html que queremos que sea el que se muestre por defecto al abrir la ventana de ayuda.
- Un *<mapref>* en el que se indica cual es el fichero mapa de *JavaHelp*.

Ahora vamos con los *<view>*

- Hemos puesto un primer *<view>* para la tabla de contenidos. En él indicamos un *<label>* que será el tooltip que salga si dejamos el ratón un rato sobre la pestaña, un *<type>* que es la clase de *JavaHelp* que tratará el contenido de esta pestaña y un *<data>* donde se dice cual es el fichero de la tabla de contenidos.
- Un segundo *<view>* es para el índice. Los tags por debajos son iguales que el anterior, pero cambia la clase *JavaHelp* que trata la pestaña y el *<data>* es ahora el fichero indice.
- Y un tercer *<view>* para la pestaña de búsqueda. Nuevamente los tres tags por debajo son iguales, pero vuelve a cambiar la clase de *JavaHelp* encargada de esta pestaña y el *<data>* en este caso es el nombre del subdirectorio que nos creo *jhindexer.jar*, con la base de datos de búsqueda, es decir, el directorio *JavaHelpSearch*.

Este fichero, *help_set.hs*, se guarda en el directorio *help*, junto a los otros de configuración de *JavaHelp*.

El código java para manejar JavaHelp.

Aunque *JavaHelp* nos permite muchas florituras, como obtener cada una de las pestañas de ayuda por separado para integrarlas en nuestra aplicación como queramos, vamos aquí a hacer un uso muy simple. Usaremos la ventana por defecto con el número mínimo de líneas posibles de código.

Lo primero que tenemos que hacer en nuestro código java, es leer el fichero de configuración *help_set.hs*. El código para ello puede ser este.

```
// Carga el fichero de ayuda
File fichero = new File("../help/help_set.hs");
URL hsURL = fichero.toURI().toURL();

// Crea el HelpSet
HelpSet helpset = new HelpSet(getClass().getClassLoader(), hsURL);
```

En primer lugar construimos en formato URL el path del fichero. Lo de "../help/help_set.hs" es en mi ejemplo concreto. Tú pon el path que te toque a tí.

Una vez que tenemos el fichero en formato URL, instanciamos la clase *HelpSet*, pasándole un *ClassLoader* y la URL del fichero. Lo del ClassLoader es importante porque el fichero *help_set.hs* podría estar dentro de nuestro jar o bien en un fichero suelto en el disco. En general, bastará con obtener el *ClassLoader* con *getClass().getClassLoader()*, como hemos hecho en el trozo de código. La clase *HelpSet* de alguna forma contiene todos los datos relativos a nuestra ayuda, leídos de los ficheros de configuración que creamos antes.

Una vez que tenemos el *HelpSet*, para hacerlo fácil, obtenemos un *HelpBroker*. Esta clase *HelpBroker* no es más que una clase de ayuda que nos ofrece *JavaHelp* para facilitarnos un uso por defecto de la ayuda desde código. Para obtener el *HelpBroker*

```
HelpBroker hb = helpset.createHelpBroker();
```

Lo difícil ya está hecho. Ahora nos queda lo fácil y pesado. Ir por todas nuestras ventanas y botones de ayuda diciendoles que muestren la ayuda. Para ello usamos los métodos *enableHelpOnButton()* y *enableHelpKey()* del *HelpBroker*. En nuestra aplicación hay un *JMenuItem* que muestra ayuda y dos ventanas. Queremos que al hacer click en el *JMenuItem* salga la ayuda y también que salga al pulsar F1 en cada una de las dos ventanas.

El código para conseguir esto es

```
// Ayuda al hacer click en el JMenuItem itemAyuda.
hb.enableHelpOnButton(itemAyuda, "aplicacion", helpset);

// Ayuda al pulsar F1 sobre la ventana principal
hb.enableHelpKey(principal.getContentPane(), "ventana_principal", helpset);

// Ayuda al pulsar F1 sobre la ventana secundaria
hb.enableHelpKey(secundaria.getContentPane(), "ventana_secundaria", helpset);
```

Unas pequeñas explicaciones sobre ese código:

- *itemAyuda* es el *JMenuItem* etiquetado "Ayuda" que debe mostrar la ventana de ayuda. Al pulsarlo, mostrará la página html cuya clave en el fichero mapa de *JavaHelp* es "*aplicacion*".
- Al método *enableHelpKey()* conviene pasarle paneles mejor que Windows, por ello se le pasa el *getContentPane()* de las ventanas. *principal* es nuestro *JFrame* principal de la aplicación y *secundaria* es un JDialog secundario de nuestra aplicación. "*ventana_principal*" y "*ventana_secundaria*" son las claves del fichero mapa de JavaHelp para indicar qué ficheros html deben mostrarse cuando se pulse F1 en esas ventanas.

El código de ejemplo de JavaHelp

Y ya está todo. Para ver el ejemplo completo, descárgate Ejemplo-JavaHelp-1.0-bin.zip. Si lo desempaquetas, tienes:

- Directorio bin, con el jar de nuestra aplcación y el de JavaHelp.
- Directorio help, con todos los ficheros de configuración, ficheros html de ayuda y la base de datos de búsqueda JavaHelpSearch.
- Directorio src, con los fuentes java de nuestro ejemplo.

Si te vas al directorio bin, puedes ejecutar y ver el ejemplo con

```
$ java -jar Ejemplo-JavaHelp-1.0.jar
```

Algunas consideraciones

Esto es un ejemplo básico y tonto para empezar con *JavaHelp*. *JavaHelp* admite muchísima más configuración. Cada uno de los ficheros de configuración aquí mencionados admite muchísimos más tipos de tags e incluso en los ficheros html se pueden meter cosas extras. Podemos así elegir iconos que queremos que se muestren, mostrar más pestañas, hacer que aparezcan ventanas "popup" en la ayuda, etc.

Si quieres más detalles, vete a la página de Sun y bájate su "JavaHelp user guide".

Estadísticas y comentarios

Numero de visitas desde el 4 Feb 2007:

- Esta pagina este mes: 143
- Total de esta pagina: 63669
- Total del sitio: 50548976

Aviso Legal